

CURSO 4: TALLER PRÁCTICO & PROYECTO FINAL INTEGRADOR

CLASE 03 • NIVEL 4 - INTEGRADOR

Clase 03: Persistencia de Datos: Modelado SQL y Transacciones ACID

«La Base de Datos como una Bóveda Acorazada para la Información»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 03** del **Curso 4: Taller Práctico & Proyecto Final Integrador**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

🎯 Objetivos de la Sesión

Competencia Conceptual: Comprender propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad), SQL parametrizado y migraciones.

Competencia Práctica: Implementar un repositorio de persistencia seguro en SQLite/PostgreSQL previniendo inyecciones SQL.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 03: Persistencia de Datos: Modelado SQL y Transacciones ACID

La persistencia de datos garantiza que la información de los usuarios permanezca intacta tras apagar o reiniciar el servidor.

☀ Metáfora Central: La Base de Datos como una Bóveda Acorazada para la Información

Una transacción ACID es como una transferencia bancaria: o se descuenta de una cuenta y se acredita en la otra, o se cancela todo.

Principios Teóricos y Modelo Mental

Inyección SQL: La vulnerabilidad #1 de bases de datos. Ocurre al concatenar strings en consultas.

Consultas parametrizadas: Separan el código SQL de los datos proporcionados por el usuario.

⚡ Regla de Oro en Python

NUNCA concatenes variables en consultas SQL; usa siempre placeholders (?, %s o :val).

Arquitectura y Movimiento de Datos en Memoria

Ciclo de vida de una transacción con Commit / Rollback automático.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Apertura de conexión y bloque transaccional (with conn:).	Transacción iniciada en BD.
2. Evaluación	Ejecución de sentencias DML (INSERT, UPDATE, DELETE).	Cambios en buffer transaccional.
3. Transformación	¿Ocurrió algún error? Sí -> Rollback / No -> Commit.	Persistencia en disco confirmada.
4. Retorno / Salida	Cierre seguro de la conexión.	Pool liberado.

Visualización Mental

El context manager 'with sqlite3.connect' ejecuta commit automáticamente si no hay excepciones.

Código de Demostración en Python 3.10+

Capa de persistencia relacional con consultas parametrizadas:

main.py (Python 3.10+)

PEP 8 Compliant

```
import sqlite3

class RepositorioUsuarios:
    def __init__(self, db_path: str = "app.db"):
        self.db_path = db_path
        self._crear_tabla()

    def _crear_tabla(self):
        with sqlite3.connect(self.db_path) as conn:
            conn.execute("""
                CREATE TABLE IF NOT EXISTS usuarios (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    nombre TEXT NOT NULL,
                    email TEXT UNIQUE NOT NULL
                )
            """)

    def insertar(self, nombre: str, email: str) -> int:
        with sqlite3.connect(self.db_path) as conn:
            cursor = conn.cursor()
            cursor.execute("INSERT INTO usuarios (nombre, email) VALUES (?, ?)", (nombre,
email))

            return cursor.lastrowid

repo = RepositorioUsuarios(":memory:")
uid = repo.insertar("Wisrovi Developer", "wisrovi@dev.com")
print(f"Usuario insertado con ID: {uid}")
```

Análisis de Ingeniería del Código

Uso de consultas parametrizadas con '?' para evitar SQL Injection y context manager seguro.

Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Vulnerabilidad de inyección SQL.

⚠ Gotcha Frecuente (Trampa de Principiante)

Formatear strings con f-strings en SQL permite a atacantes ejecutar comandos destructivos (ej. ' OR 1=1; DROP TABLE...).

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
cursor.execute(f'SELECT * FROM users WHERE email = \'{email}\')
```

 # ❌ Vulnerable a SQL Injection

✅ Patrón Correcto

```
cursor.execute('SELECT * FROM users WHERE email = ?', (email,))
```

 # ✅ 100% Seguro

🛡 Consejo de Resiliencia en Producción

Usa SQLAlchemy o SQLModel para proyectos de gran escala con mapeo objeto-relacional.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 03: Persistencia de Datos: Modelado SQL y Transacciones ACID**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	La Base de Datos como una Bóveda Acorazada para la Información
2. Regla de Oro	NUNCA concatenes variables en consultas SQL; usa siempre placeholders (?, %s o :val).
3. Gotcha Crítico	Formatear strings con f-strings en SQL permite a atacantes ejecutar comandos destructivos (ej. ' OR 1=1; DROP TABLE...).
4. Buenas Prácticas	Usa SQLAlchemy o SQLModel para proyectos de gran escala con mapeo objeto-relacional.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Crea una tabla 'pedidos' vinculada por clave foránea (FOREIGN KEY) a la tabla de usuarios.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.